

LAB # 9

Explain and Reproduce the Working of 16x2 LCD on STM32F407 Trainer Board using STM32CubeIDE and HAL Library

Objectives

- Understand and demonstrate the interfacing of a 16x2 LCD with the STM32F407 microcontroller using the trainer board
- Develop and modify programs using HAL library functions to control LCD displays
- Master LCD programming techniques for displaying dynamic text and scrolling messages

Pre-Lab Exercise

Read the information provided below to understand the fundamental operations of LCD technology and its interfacing with microcontrollers. LCDs can be configured to perform various tasks through different interfacing combinations and programming techniques.

Introduction

In this lab, you'll explore the operation modes of LCD displays and learn how to program and interface a 16x2 LCD with the STM32F407 microcontroller.

Why LCDs?

LCD displays have become the go-to choice for electronics projects, providing visual feedback and status information. The 16x2 alphanumeric LCD (Figure 9-1) is one of the most popular display modules available today, offering an excellent balance of functionality, cost, and ease of use.



Figure 9 1: 16 x 2 LCD

Why Choose LCD Over LED?

In recent years, LCDs have increasingly replaced traditional LEDs in embedded systems due to several compelling advantages:

- **Cost-Effective:** Declining prices make LCDs accessible for all project scales
- **Versatility:** Ability to display numbers, characters, and graphics—far beyond the limited character set of LEDs
- **User-Friendly Programming:** Simple commands for displaying characters and graphics

- **Professional Appearance:** Clean, modern look suitable for consumer products

LCD Architecture and Registers

A 16x2 LCD features two horizontal lines, each capable of displaying 16 characters. The display is controlled through two internal registers:

1. Command Register

- Used to send special instructions to the LCD, such as:
- Clearing the screen
- Positioning the cursor
- Setting display modes
- Controlling cursor behavior

2. Data Register

- Used to write actual characters and symbols that appear on the display.

Understanding 4-bit vs 8-bit Mode

The HD44780 LCD controller (standard in most 16x2 displays) supports two communication modes:

1. 8-bit Mode (Used in this Lab)

- Uses all 8 data lines (D0-D7)
- Faster communication
- Requires more GPIO pins

2. 4-bit Mode

- Uses only 4 data lines (D4-D7)
- Reduces GPIO pin usage—critical in resource-constrained applications
- Slightly more complex but highly efficient
- Commands and data are sent as two 4-bit "nibbles"

Important: The operation mode is not defined at power-up. You must explicitly configure the LCD for 4-bit mode by sending the Function Set command three times during initialization.

LCD Control Pins Explained

1. Register Select (RS) Pin

- LOW (0): Command mode—send instructions to the LCD
- HIGH (1): Data mode—send characters to display

2. Read/Write (R/W) Pin

- LOW (0): Write operation—send data to LCD
- HIGH (1): Read operation—read data from LCD (*Most applications only use write mode*)

3. Enable (E) Pin

- Triggers the actual read/write process
- Data is latched on the high-to-low transition of this pin
- Must be pulsed for every operation

Pin Configuration

Table 9-1: LCD Pin Functions and STM32 Connections

Pin No.	Function Name	Description	STM32 Pin
1	Ground	0V reference	GND
2	VCC	Supply voltage (5V, range: 4.7V – 5.3V)	VCC
3	VEE/Contrast	Contrast adjustment via potentiometer	Variable
4	RS	Register Select (0=Command, 1=Data)	PB1
5	R/W	Read/Write (0=Write, 1=Read)	PB0
6	E	Enable (High-to-low pulse triggers operation)	PE7
7	DB0	Data bit 0 (8-bit mode only)	PE8
8	DB1	Data bit 1 (8-bit mode only)	PE9
9	DB2	Data bit 2 (8-bit mode only)	PE10
10	DB3	Data bit 3 (8-bit mode only)	PE11
11	DB4	Data bit 4	PE12
12	DB5	Data bit 5	PE13
13	DB6	Data bit 6	PE14
14	DB7	Data bit 7	PE15
15	LED+	Backlight positive (5V)	VCC
16	LED-	Backlight negative	GND

Essential LCD Commands

Table 9-2: Common LCD Command Instructions

Hex Code	Command	Description
0x01	Clear Display	Clears entire display and returns cursor to home
0x02	Return Home	Returns cursor to position (0,0)
0x04	Entry Mode	Decrement cursor (shift left)
0x06	Entry Mode	Increment cursor (shift right)
0x05	Display Shift	Shift entire display right
0x07	Display Shift	Shift entire display left
0x08	Display OFF	Turn off display and cursor
0x0C	Display ON	Display ON, cursor OFF
0x0E	Display ON	Display ON, cursor blinking
0x0F	Display ON	Display ON, cursor ON
0x38	Function Set	8-bit mode, 2 lines, 5×7 matrix
0x28	Function Set	4-bit mode, 2 lines, 5×7 matrix
0x80	Set DDRAM	Force cursor to the beginning of first line
0xC0	Set DDRAM	Force cursor to the beginning of second line
0x83	Set DDRAM	Cursor to line 1, position 3
0xC1	Set DDRAM	Cursor to line 2, position 1
0xC2	Set DDRAM	Cursor to line 2, position 2

How to Program an LCD: The Complete Process

Programming an LCD involves a specific sequence of operations. Understanding this process is crucial for successful LCD interfacing.

1. Initialization Sequence

The LCD must be properly initialized before use. This process ensures the display is in a known state and ready to receive commands and data.

Sending Command to LCD - Steps

1. **Start:** Begin the process of sending an LCD command.
2. **Set RS (Register Select) Pin Low:** Set the RS pin to a LOW state (logic 0). This indicates that the data being sent is a command, not character data.
3. **Set R/W (Read/Write) Pin Low:** Set the R/W pin to a LOW state (logic 0). This indicates that data is being written to the LCD.
4. **Send 8-bit Command:** Send the command byte to the LCD's data pins.
5. **Pulse Enable (E) Pin:**
 - a. Bring the E pin HIGH.
 - b. Introduce a short delay.
 - c. Bring the E pin LOW.
6. **End:** The command sending process is complete.

Algorithm 1: Writing a Command to LCD

```
ALGORITHM: Send_Command_to_LCD(command)
INPUT: command (8-bit command byte)
OUTPUT: Command executed on LCD

BEGIN
  1. Set RS = 0           // Select Command Register
  2. Set R/W = 0          // Write mode
  3. Put 8-bit command on data pins D0-D7
  4. Set E = 1           // Enable HIGH
  5. Delay 1ms           // Hold time
  6. Set E = 0           // Enable LOW (latch data)
  7. Delay 2ms           // Command execution time
END
```

Sending Data to LCD - Steps

1. **Start:** Begin the process of sending an LCD command.
2. **Set RS (Register Select) Pin Low:** Set the RS pin to a HIGH state (logic 1). This indicates that the data being sent is not a command.
3. **Set R/W (Read/Write) Pin Low:** Set the R/W pin to a LOW state (logic 0). This indicates that data is being written to the LCD.
4. **Send 8-bit Data:** Send the Data byte to the LCD's data pins.
5. **Pulse Enable (E) Pin:**
 - a. Bring the E pin HIGH.
 - b. Introduce a short delay.
 - c. Bring the E pin LOW.

End: The command sending process is complete

Algorithm 2: Writing Data (Characters) to LCD

```
ALGORITHM: Send_Data_to_LCD(character)
INPUT: character (8-bit ASCII character)
OUTPUT: Character displayed on LCD

BEGIN
  1. Set RS = 1          // Select Data Register
  2. Set R/W = 0         // Write mode
  3. Put 8-bit character on data pins D0-D7
  4. Set E = 1           // Enable HIGH
  5. Delay 1ms           // Hold time
  6. Set E = 0           // Enable LOW (latch data)
  7. Delay 2ms           // Character write time
END
```

Algorithm 3: Displaying a String

```
ALGORITHM: Display_String_on_LCD(string, position)
INPUT: string (array of characters), position (cursor location)
OUTPUT: String displayed on LCD at specified position

BEGIN
  1. Send_Command_to_LCD(position) // Set cursor position
                                     // 0x80 for line 1
                                     // 0xC0 for line 2
  2. i = 0
  3. WHILE string[i] != '\0' DO
      4. Send_Data_to_LCD(string[i])
      5. i = i + 1
    END WHILE
  6. RETURN
END
```

Key Programming Insights

The Enable Pin is Critical

The Enable (E) pin acts as a "latch" signal. Data is captured by the LCD on the falling edge (high-to-low transition) of the Enable pin. Always ensure proper timing:

- Hold E HIGH for at least 450ns
- Hold E LOW for at least 450ns

Timing Matters

Different commands require different execution times:

- Clear Display: ~2ms
- Return Home: ~2ms
- Other commands: ~40µs
- Data write: ~40µs

RS Pin Determines Mode

This single pin controls whether you're sending:

- Commands (RS=0): Instructions like clear, cursor position
- Data (RS=1): Actual characters to display

In-Lab Exercise

Task 1: Display a String on LCD

Complete the program that will display a string on the LCD using 8-bit mode.

```
// LCD Pin Definitions
#define RS_PIN GPIO_PIN_1
#define RS_PORT GPIOB
#define RW_PIN GPIO_PIN_0
#define RW_PORT GPIOB
#define EN_PIN GPIO_PIN_7
#define EN_PORT GPIOE

// Data pins D0-D7 on PE8-PE15
#define LCD_DATA_PORT GPIOE

// Function prototypes
void LCD_Init(void);
void LCD_Command(uint8_t cmd);
void LCD_Data(uint8_t data);
void LCD_String(char *str);
void LCD_Clear(void);
void LCD_Set_Cursor(uint8_t row, uint8_t col);

void LCD_Enable_Pulse(void) {
    HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_RESET);
    HAL_Delay(1);
}

void LCD_Write_8bit(uint8_t data) {
    // Write 8-bit data to D0-D7 (PE8-PE15)
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, (data&0x01)?GPIO_PIN_SET:GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9, (data&0x02)?GPIO_PIN_SET:GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_10, (data&0x04)?GPIO_PIN_SET:GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_11, (data&0x08)?GPIO_PIN_SET:GPIO_PIN_RESET);
    Do the same for other pins
}

void LCD_Command(uint8_t cmd) {
```

```

HAL_GPIO_WritePin(RS_PORT, RS_PIN, GPIO_PIN_RESET); // RS = 0 for command
HAL_GPIO_WritePin(RW_PORT, RW_PIN, GPIO_PIN_RESET); // RW = 0 for write

LCD_Write_8bit(cmd); // Send 8-bit command
LCD_Enable_Pulse(); // Pulse enable pin
HAL_Delay(2); // Wait for command execution
}

void LCD_Data(uint8_t data) {
    HAL_GPIO_WritePin(RS_PORT, RS_PIN, GPIO_PIN_SET); // RS = 1 for data
    HAL_GPIO_WritePin(RW_PORT, RW_PIN, GPIO_PIN_RESET); // RW = 0 for write

    LCD_Write_8bit(data); // Send 8-bit data
    LCD_Enable_Pulse(); // Pulse enable pin
    HAL_Delay(2); // Wait for data write
}

void LCD_Init(void) {
    HAL_Delay(50); // Wait for LCD to power up (>15ms)

    // Initialization sequence for 8-bit mode
    LCD_Command(0x30); // Function set: 8-bit mode
    HAL_Delay(5);
    LCD_Command(0x30); // Function set: 8-bit mode
    HAL_Delay(1);
    LCD_Command(0x30); // Function set: 8-bit mode
    HAL_Delay(1);

    // Function Set: 8-bit, 2 lines, 5x7 font
    LCD_Command(0x38);

    // Display ON, Cursor OFF
    LCD_Command(0x0C);

    // Clear Display
    LCD_Command(0x01);
    HAL_Delay(2);

    // Entry Mode: Increment cursor, no display shift
    LCD_Command(0x06);
}

void LCD_String(char *str) {
    while(*str) {
        LCD_Data(*str++);
    }
}

void LCD_Clear(void) {
    LCD_Command(0x01);
    HAL_Delay(2);
}

void LCD_Set_Cursor(uint8_t row, uint8_t col) {
    uint8_t address = (row == 0) ? 0x80 : 0xC0;
    address += col;
    LCD_Command(address);
}

void SystemClock_Config(void);

```

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    LCD_Init();

    // Display on first line
    LCD_Set_Cursor(0, 0);
    LCD_String("Hello, STM32!");

    // Display on second line
    LCD_Set_Cursor(1, 0);
    LCD_String("Lab 09 - LCD");

    while(1) {
        // Main loop
    }
}
```

Task 2: Scrolling Text Display

Write a complete C program that displays two strings (e.g., Your Name and Your Registration Number) on the LCD such that:

- First message: Scrolls from left to right on the upper line
- Second message: Scrolls from right to left on the lower line

Hint: Use `LCD_Command(0x18)` to shift display left and `LCD_Command(0x1C)` to shift display right, combined with appropriate delays for smooth animation.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____

Date: _____